Day 70 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 1 reference

Watch: Attention: Problems with Seq2seq Models (Greg Durrett, UT Austin) | S18 Sequence to Sequence models: Attention Models (Carnegie Mellon University Deep Learning)

`day70_information_bottleneck.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula; the green **SYSTEM DIAGRAM** and the graph are the ones the script saved. Day 69's model, LSTM and data code are reused verbatim and not repeated here. A glossary is at the end.

MATH BOX LEGEND

Symbols: n = number of input digits · k = number of digits to write back · H = hidden size; the decoder receives 2H numbers (h and c) · exact match = every output token right

Segment 0 — Setup

Settings

CODE

```
SEEDS = [0, 1, 2]
EVAL_LENGTHS = (4, 8, 12, 16, 20)
HIDDENS = (16, 32, 64)
KS = (4, 8, 12, 16, 20)
STEPS_MAIN = 40000
STEPS_K = 30000
STEPS_LONG = 120000
EVERY = 5000
```

- `EVAL_LENGTHS = (4, 8, 12, 16, 20)` — a tuple: the test lengths, in a fixed order the tables follow.
- `STEPS_LONG = 120000` — three times STEPS_MAIN, for Section 6's longer training.
- `EVERY = 5000` — test accuracy is recorded every 5,000 steps, so the curves have 8, 6 or 24 points (40,000, 30,000 or 120,000 steps).

Segment 1 — Data and training

make_batch_first: the new task

CODE

```
def make_batch_first(rng, m, n, k):
    src = rng.randint(1, 9, size=(m, n))
    tgt = src[:, :k]
    dec_in = np.concatenate([np.full((m, 1), GO), tgt], axis=1)
    dec_out = np.concatenate([tgt, np.full((m, 1), EOS)], axis=1)
    return np.eye(V)[src], np.eye(V)[dec_in], dec_out, src
```

- `tgt = src[:, :k]` — slicing the first k columns: the only difference from Day 69's COPY.
- `rng.randint(1, 9, size=(m, n))` — the same random draw as make_batch, so the inputs match a COPY batch from the same rng.
- `np.eye(V)[src]` — one-hot vectors by indexing rows of the identity matrix (Day 69).

fixed_test and exact_and_token

CODE

```
def fixed_test(n, task, k=None, m=300):
    rng = np.random.RandomState(12345 + n)
    return make_batch(rng, m, n, task) if k is None else make_batch_first(rng, m, n, k)
def exact_and_token(net, batch):
    Xs, _, Y, _ = batch
    pred = net.generate(Xs, Y.shape[1])
    return np.all(pred == Y, axis=1).mean(), (pred == Y).mean()
```

- `np.random.RandomState(12345 + n)` — a separate fixed seed per length: every model meets the same test sequences.
- `... if k is None else ...` — a conditional expression picks COPY/REVERSE or the first-k task.
- `net.generate(Xs, Y.shape[1])` — greedy decoding for as many steps as the target has (digits + EOS).
- `np.all(pred == Y, axis=1)` — True where the whole sequence is right.

train and train_cached

CODE

```
def train(task, hidden, seed, steps, lo, hi, k=None, every=EVERY, eval_lengths=EVAL_LENGTHS):
    net = Seq2Seq(hidden=hidden, seed=seed)
    rng = np.random.RandomState(seed + 100)
    tests = {n: fixed_test(n, task, k) for n in eval_lengths}
    curve = []
    for step in range(1, steps + 1):
        n = rng.randint(lo, hi + 1)
        if k is None:
            Xs, Xd, Y, _ = make_batch(rng, 32, n, task)
        else:
            Xs, Xd, Y, _ = make_batch_first(rng, 32, n, min(k, n))
        net.train_step(Xs, Xd, Y)
        if step % every == 0:
            curve.append([exact_and_token(net, tests[n])[0] for n in eval_lengths])
    return net, np.array(curve)
RUNS = {}
def train_cached(*args, **kwargs):
    key = (args, tuple(sorted(kwargs.items())))
    if key not in RUNS:
        RUNS[key] = train(*args, **kwargs)
    return RUNS[key]
```

- `rng.randint(lo, hi + 1)` — randint excludes its upper end, so hi + 1 makes hi possible.
- `min(k, n)` — a short input cannot give back more digits than it has.
- `curve.append([...])` — one row per checkpoint: exact match at each test length.
- `def train_cached(*args, **kwargs)` — *args collects positional arguments, **kwargs keyword ones; together they form the cache key.

Segment 2 — The experiments

Section 2: the fair test

CODE

```
for hidden in HIDDENS:
    curves = [train_cached("copy", hidden, s, STEPS_MAIN, 3, 20)[1] for s in SEEDS]
    final[hidden] = np.mean([c[-1] for c in curves], axis=0)
    print(f"{hidden:>7}{2 * hidden:>12}" + "".join(f"{e:>8.3f}" for e in final[hidden]) +
        f"{len(SEEDS):>8}")
```

- `c[-1] for c in curves` — the last checkpoint of each seed's curve: the final model.

- `np.mean([...], axis=0)` — average over seeds, length by length.
- `2 * hidden` — the decoder starts from h AND c, so it receives twice the hidden size.

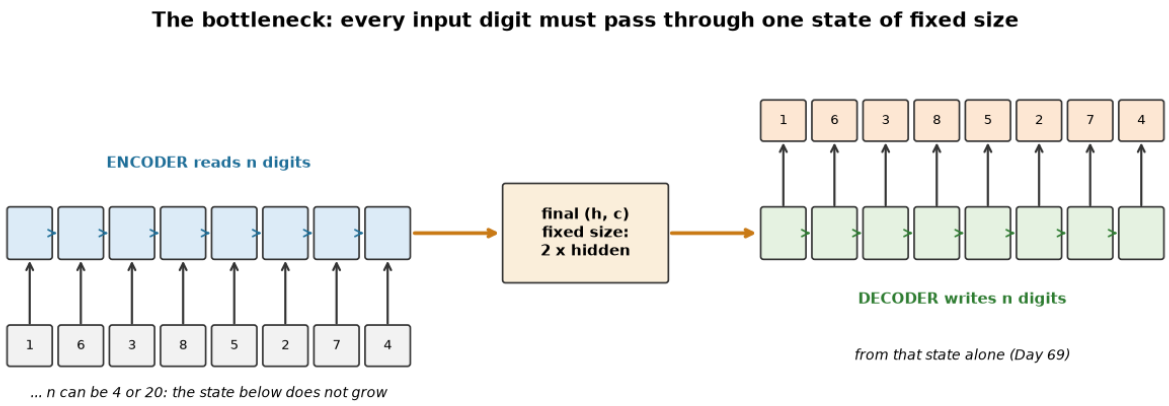
Section 3: same distance, more to hold

CODE

```
for k in KS:
    runs = [train_cached("first", 32, s, STEPS_K, 20, 20, k=k, eval_lengths=(20,)) for s in SEEDS]
    scores = [exact_and_token(net, fixed_test(20, "first", k)) for net, _ in runs]
    res[k] = np.mean(scores, axis=0)
    print(f"{k:>4}{res[k][0]:>13.3f}{res[k][1]:>11.3f}{'  '.join(f'{e:.3f}' for e, _ in
        scores):>30}")
```

- `train_cached("first", 32, s, STEPS_K, 20, 20, k=k, ...)` — lo = hi = 20: every training input has exactly 20 digits.
- `eval_lengths=(20,)` — a one-element tuple needs the comma.
- `np.mean(scores, axis=0)` — averages the (exact, token) pairs over the seeds.

SYSTEM DIAGRAM



The bottleneck: n digits in, one state of fixed size, n digits out.

Segment 3 — The probe

probe_features and fit_probe

MATH

$$p = softmax((F - \mu) / \sigma \cdot W + b);\quad W \leftarrow W - lr \cdot Z^T (p - onehot(y)) / m$$

CODE

```
def probe_features(net, Xs):
    h, c, _ = net.encode(Xs)
    return np.concatenate([h, c], axis=1)
def fit_probe(F, y, iters=400, lr=0.5):
    mu, sd = F.mean(axis=0), F.std(axis=0) + 1e-8
    Z = (F - mu) / sd
    W = np.zeros((F.shape[1], V)); b = np.zeros(V)
    for _ in range(iters):
        p = softmax(Z @ W + b)
        p[np.arange(len(y)), y] -= 1
        W -= lr * Z.T @ p / len(y); b -= lr * p.mean(axis=0)
    return lambda G: ((G - mu) / sd @ W + b).argmax(axis=1)
```

- `np.concatenate([h, c], axis=1)` — the probe sees exactly what the decoder receives.
- `(F - mu) / sd` — standardising: each feature gets mean 0 and spread 1, so one learning rate suits all.
- `p[np.arange(len(y)), y] -= 1` — softmax + cross-entropy gradient, as in every classifier of this series.
- `return lambda G: ...` — the function returns another function: a trained predictor for new features.

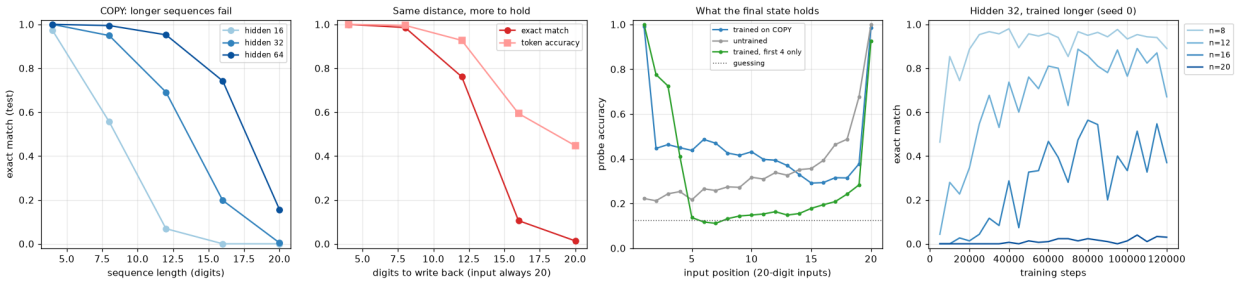
Running one probe per input position

CODE

```
for name, net in models.items():
    Ftr, Fte = probe_features(net, Xtr), probe_features(net, Xte)
    res[name] = [(fit_probe(Ftr, src_tr[:, p])(Fte) == src_te[:, p]).mean() for p in range(20)]
```

- `fit_probe(Ftr, src_tr[:, p])(Fte)` — train a probe for position p, then call the returned function on the test features.
- `for p in range(20)` — 20 separate probes, one per input position.
- `Seq2Seq(hidden=32, seed=0)` — an untrained model as a baseline: what a random encoder keeps.

REAL GRAPH



Real graph from the script: length, amount, the probe, and longer training.

Quick Syntax Glossary — new or reinforced constructs

- `src[:, :k]` — the first k columns of every row.
- `rng.randint(lo, hi + 1)` — an integer from lo to hi inclusive.
- `(20,)` — a tuple with one element.

- `def f(*args, **kwargs)` — accept any positional and keyword arguments.
- `lambda G: ...` — a small unnamed function, here returned by another function.
- `x.std(axis=0)` — the spread of each column.
- `a if cond else b` — a conditional expression.

End of Day 70 syntax reference. For the experiments and what they showed, see the Day 70 complete guide.